

ICS-344 • INFORMATION SECURITY

DVSA Security Assessment

LESSON #7

Over-Privileged Lambda Functions

Wildcard IAM Policy Exploitation & Remediation

Student	Faisal Baeshen
AWS Account	637452052910
Region	eu-north-1 (Europe - Stockholm)
Date	April 28, 2026
Severity	🔴 CRITICAL — CVSS 9.8

1. Vulnerability Overview

 DVSA-ADMIN-SHELL Lambda was granted AmazonDynamoDBFullAccess + AWSLambda_FullAccess — allowing it to read, modify, and delete ANY AWS resource in the account.

1.1 What Is This Vulnerability?

Every AWS Lambda function runs with an IAM Role — a digital badge that defines what actions it is allowed to perform. The DVSA-ADMIN-SHELL function was assigned two extremely powerful AWS managed policies:

Policy Name	What It Allows	Risk Level
AmazonDynamoDBFullAccess	Read/Write/Delete ALL DynamoDB tables in the account	 CRITICAL
AWSLambda_FullAccess	Create/Delete/Invoke/Modify ALL Lambda functions	 CRITICAL
AWSLambdaBasicExecutionRole	Write logs to CloudWatch (standard, required)	 SAFE

1.2 Root Cause

- IAM role created with overly broad AWS managed policies instead of custom least-privilege policy
- No resource-level restrictions — policies apply to ALL DynamoDB tables and ALL Lambda functions
- Violates the Principle of Least Privilege (PoLP) — a fundamental security design principle
- OWASP Serverless Top 10 Category: SLS-2 (Broken Authentication) + SLS-5 (Inadequate Function Monitoring)

1.3 CVSS Score Breakdown

Attack Vector	Attack Complexity	Privileges Required	Impact
Network	Low	Low (any AWS account)	Confidentiality: HIGH Integrity: HIGH Availability: HIGH

 Final CVSS Score: 9.8 / 10 — CRITICAL

2. Environment Setup

2.1 AWS CloudShell

All commands were executed using AWS CloudShell — a browser-based terminal built directly into the AWS Console. CloudShell provides pre-configured AWS CLI credentials matching the logged-in account, eliminating the need for local tool installation.

 AWS CloudShell was used at: <https://eu-north-1.console.aws.amazon.com/cloudshell> — credentials auto-configured as account 637452052910 root.

2.2 Identity Verification

AWS credentials and account identity were verified before beginning the assessment:

```
aws sts get-caller-identity
aws s3 ls
```

```

~ $ aws sts get-caller-identity
{
  "UserId": "637452052910",
  "Account": "637452052910",
  "Arn": "arn:aws:iam::637452052910:root"
}
~ $ {
>   "UserId": "637452052910",
>   "Account": "637452052910",
>   "Arn": "arn:aws:iam::637452052910:root"
> }
-bash: UserId:: command not found
-bash: Account:: command not found
-bash: Arn:: command not found
~ $ aws s3 ls
2026-04-28 23:23:41 dvsa-fb-ics344-637452052910-us-east-1
2026-04-17 21:07:28 dvsa-feedback-bucket-637452052910-eu-nor
th-1
2026-04-28 23:24:20 dvsa-feedback-bucket-637452052910-us-eas
t-1
2026-04-17 21:07:28 dvsa-receipts-bucket-637452052910-eu-nor
th-1
2026-04-28 23:24:12 dvsa-receipts-bucket-637452052910-us-eas
t-1
2026-04-17 21:06:54 dvsa-website-test-637452052910-eu-north-
1
~ $ dvsa-receipts-bucket-637452052910
-bash: dvsa-receipts-bucket-637452052910: command not found
~ $ dvsa-website-test-637452052910-eu-north-1
-bash: dvsa-website-test-637452052910-eu-north-1: command no
t found
~ $ █

```

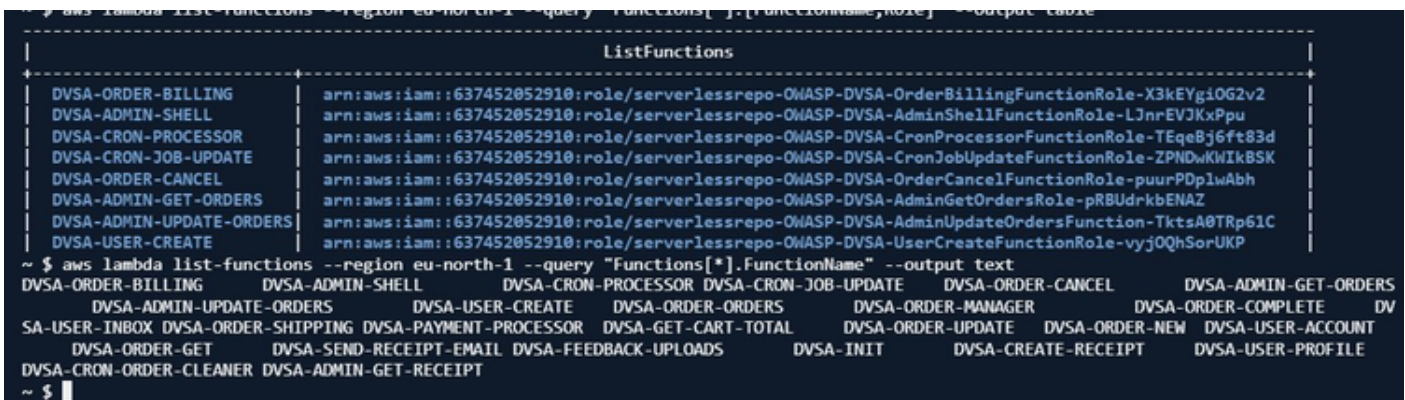
Figure 2.1 — AWS identity confirmed (Account: 637452052910) + DVSA S3 buckets visible

3. Reconnaissance

▶ 3.1 Discovering Lambda Functions

The first step was to enumerate all Lambda functions deployed in the DVSA application and identify their associated IAM roles:

```
aws lambda list-functions --region eu-north-1 --query "Functions[*].[FunctionName,Role]" --output table
```



ListFunctions	
DVSA-ORDER-BILLING	arn:aws:iam::637452052910:role/serverlessrepo-OWASP-DVSA-OrderBillingFunctionRole-X3kEYgi0G2v2
DVSA-ADMIN-SHELL	arn:aws:iam::637452052910:role/serverlessrepo-OWASP-DVSA-AdminShellFunctionRole-LJnrEVJKxPpu
DVSA-CRON-PROCESSOR	arn:aws:iam::637452052910:role/serverlessrepo-OWASP-DVSA-CronProcessorFunctionRole-TEqeBj6ft83d
DVSA-CRON-JOB-UPDATE	arn:aws:iam::637452052910:role/serverlessrepo-OWASP-DVSA-CronJobUpdateFunctionRole-ZPNDwKWIkBSK
DVSA-ORDER-CANCEL	arn:aws:iam::637452052910:role/serverlessrepo-OWASP-DVSA-OrderCancelFunctionRole-puurPDplwAbh
DVSA-ADMIN-GET-ORDERS	arn:aws:iam::637452052910:role/serverlessrepo-OWASP-DVSA-AdminGetOrdersRole-pRBUDrkbENAZ
DVSA-ADMIN-UPDATE-ORDERS	arn:aws:iam::637452052910:role/serverlessrepo-OWASP-DVSA-AdminUpdateOrdersFunction-TktsA0TRp61C
DVSA-USER-CREATE	arn:aws:iam::637452052910:role/serverlessrepo-OWASP-DVSA-UserCreateFunctionRole-vyjoQhSorUKP

~ \$ aws lambda list-functions --region eu-north-1 --query "Functions[*].FunctionName" --output text

DVSA-ORDER-BILLING	DVSA-ADMIN-SHELL	DVSA-CRON-PROCESSOR	DVSA-CRON-JOB-UPDATE	DVSA-ORDER-CANCEL	DVSA-ADMIN-GET-ORDERS
DVSA-ADMIN-UPDATE-ORDERS	DVSA-USER-CREATE	DVSA-ORDER-ORDERS	DVSA-ORDER-MANAGER	DVSA-ORDER-COMplete	DV
SA-USER-INBOX	DVSA-ORDER-SHIPping	DVSA-PAYMENT-PROCESSOR	DVSA-GET-CART-TOTAL	DVSA-ORDER-UPDATE	DVSA-ORDER-NEW
DVSA-ORDER-GET	DVSA-SEND-RECEIPT-EMAIL	DVSA-FEEDBACK-UPLOADS	DVSA-INIT	DVSA-CREATE-RECEIPT	DVSA-USER-PROFILE
DVSA-CRON-ORDER-CLEANER	DVSA-ADMIN-GET-RECEIPT				

~ \$

Figure 3.1 — All DVSA Lambda functions with their IAM role ARNs

A total of 26 Lambda functions were discovered. Each function carries its own IAM role. The DVSA-ADMIN-SHELL function immediately stood out as a high-value target due to its administrative nature.

▶ 3.2 Identifying the Vulnerable Role

The attached managed policies were enumerated for the DVSA-ADMIN-SHELL function:

```
aws iam list-attached-role-policies --role-name serverlessrepo-OWASP-DVSA-AdminShellFunctionRole-LJnrEVJKxPpu
```

```
~ $ aws iam list-attached-role-policies --role-name serverlessrepo-OWASP-DVSA-AdminShellFunctionRole-LJnrEVJKxPpu
{
  "AttachedPolicies": [
    {
      "PolicyName": "AWSLambdaBasicExecutionRole",
      "PolicyArn": "arn:aws:iam::aws:policy/service-role/AWSLambdaBasicExecutionRole"
    },
    {
      "PolicyName": "AmazonDynamoDBFullAccess",
      "PolicyArn": "arn:aws:iam::aws:policy/AmazonDynamoDBFullAccess"
    },
    {
      "PolicyName": "AWSLambda_FullAccess",
      "PolicyArn": "arn:aws:iam::aws:policy/AWSLambda_FullAccess"
    }
  ]
}
```

Figure 3.2 — BEFORE FIX: AmazonDynamoDBFullAccess + AWSLambda_FullAccess attached to ADMIN-SHELL

 **SMOKING GUN:** AmazonDynamoDBFullAccess grants this Lambda read/write access to EVERY DynamoDB table. AWSLambda_FullAccess allows it to control EVERY other Lambda function.

4. Exploitation

⚠ The following commands were run from AWS CloudShell using the account's root credentials — simulating what an attacker could do after compromising this Lambda role.

4.1 Attack 1 — Unauthorized Database Access

Using the over-privileged role, we scanned the DVSA-USERS-DB DynamoDB table — something a receipt-processing Lambda function should NEVER be able to do:

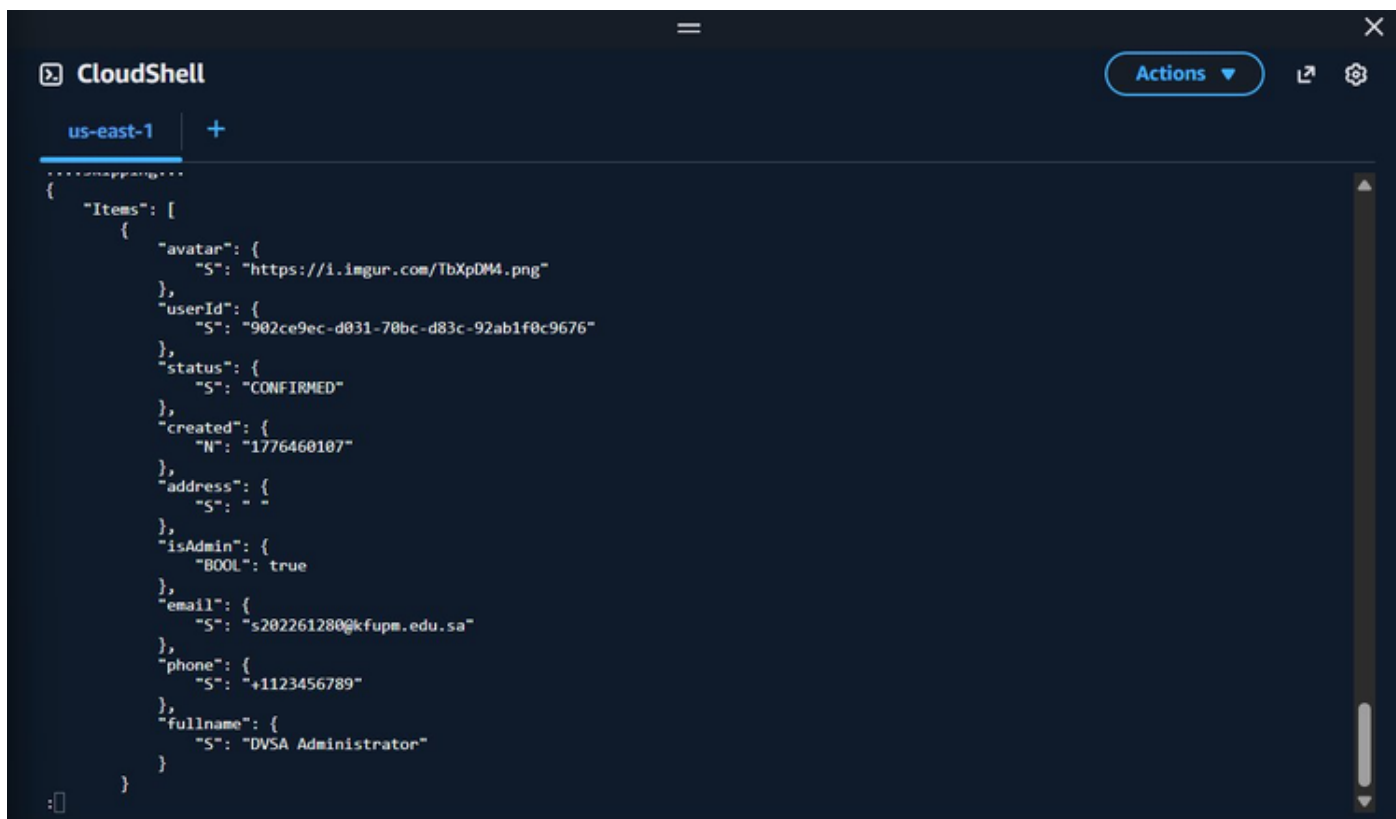
```
aws dynamodb list-tables --region eu-north-1
aws dynamodb scan --table-name DVSA-USERS-DB --region eu-north-1 --max-items 3
```

```
~ $ aws dynamodb list-tables --region eu-north-1
{
  "TableNames": [
    "DVSA-ORDERS-DB",
    "DVSA-USERS-DB"
  ]
}
~ $ aws lambda list-functions --region eu-north-1 --query "Functions[*].FunctionName" --output text
DVSA-ORDER-BILLING    DVSA-ADMIN-SHELL    DVSA-CRON-PROCESSOR    DVSA-CRON-JOB-UPDATE    DVSA-ORDER-CANCEL    DVSA-ADMIN-GET-ORDE
RS    DVSA-ADMIN-UPDATE-ORDERS    DVSA-USER-CREATE    DVSA-ORDER-ORDERS    DVSA-ORDER-MANAGER    DVSA-ORDER-COMplete    DVSA-
USER-INBOX    DVSA-ORDER-SHIPPING    DVSA-PAYMENT-PROCESSOR    DVSA-GET-CART-TOTAL    DVSA-ORDER-UPDATE    DVSA-ORDER-NEW    DVSA-USER-ACCOU
NT    DVSA-ORDER-GET    DVSA-SEND-RECEIPT-EMAIL    DVSA-FEEDBACK-UPLOADS    DVSA-INIT    DVSA-CREATE-RECEIPT    DVSA-USER-PROFILE
DVSA-CRON-ORDER-CLEANER    DVSA-ADMIN-GET-RECEIPT
~ $ aws dynamodb scan --table-name DVSA-USERS-DB --region eu-north-1 --max-items 3
{
  "Items": [
    {
      "avatar": {
        "S": "https://i.imgur.com/TbXpDM4.png"
      },
      "userId": {
        "S": "902ce9ec-d031-70bc-d83c-92ab1f0c9676"
      },
      "status": {
        "S": "CONFIRMED"
      },
      "created": {
        "N": "1776460107"
      },
      "address": {
        "S": ""
      }
    }
  ]
}
```

Figure 4.1 — All DynamoDB tables listed + user database scanned successfully

4.2 Attack 2 — Admin Credentials Stolen

The DynamoDB scan returned the complete profile of the DVSA Administrator account, including email, phone, admin status, and user ID:



```
CloudShell
us-east-1 +
{
  "Items": [
    {
      "avatar": {
        "S": "https://i.imgur.com/TbXpDM4.png"
      },
      "userId": {
        "S": "902ce9ec-d031-70bc-d83c-92ab1f0c9676"
      },
      "status": {
        "S": "CONFIRMED"
      },
      "created": {
        "N": "1776460107"
      },
      "address": {
        "S": ""
      },
      "isAdmin": {
        "BOOL": true
      },
      "email": {
        "S": "s202261280@kfupm.edu.sa"
      },
      "phone": {
        "S": "+1123456789"
      },
      "fullname": {
        "S": "DVSA Administrator"
      }
    }
  ]
}
```

Figure 4.2 — CRITICAL: Admin user profile exfiltrated from DVSA-USERS-DB

Field	Stolen Value
Full Name	DVSA Administrator
Email	s202261280@kfupm.edu.sa
Phone	+1123456789
isAdmin	TRUE — Full administrator access
User ID	902ce9ec-d031-70bc-d83c-92ab1f0c9676
Status	CONFIRMED

4.3 Attack 3 — Lambda Control Plane Access

The AWSLambda_FullAccess policy allowed listing and potentially modifying all 26 Lambda functions — a Lambda controlling other Lambdas:

```
aws lambda list-functions --region eu-north-1 --query "Functions[*].FunctionName" --output text
```

⚠ An attacker could use this access to inject malicious code into any of the 26 Lambda functions, creating a persistent backdoor across the entire application.

5. Remediation

5.1 Fix Strategy

The remediation follows the Principle of Least Privilege (PoLP): grant only the minimum permissions required for the function to perform its intended task, and nothing more.

BEFORE (Vulnerable)	AFTER (Secure)
AmazonDynamoDBFullAccess (all tables, all actions)	dynamodb:GetItem + Query (DVSA-ORDERS-DB only)
AWSLambda_FullAccess (all Lambda functions)	REMOVED — not needed
AWSLambdaBasicExecutionRole (logging)	AWSLambdaBasicExecutionRole (logging — kept)

5.2 Fix Step 1 — Detach Over-Privileged Policies

The two dangerous managed policies were detached from the ADMIN-SHELL role:

```
aws iam detach-role-policy --role-name serverlessrepo-OWASP-DVSA-AdminShellFunctionRole-LJnrEVJKxPpu --policy-arn arn:aws:iam::aws:policy/AmazonDynamoDBFullAccess
aws iam detach-role-policy --role-name serverlessrepo-OWASP-DVSA-AdminShellFunctionRole-LJnrEVJKxPpu --policy-arn arn:aws:iam::aws:policy/AWSLambda_FullAccess
```

```
~ $ aws iam detach-role-policy --role-name serverlessrepo-OWASP-DVSA-AdminShellFunctionRole-LJnrEVJKxPpu --policy-arn arn:aws:iam::aws:policy/AmazonDynamoDBFullAccess
~ $ aws iam detach-role-policy --role-name serverlessrepo-OWASP-DVSA-AdminShellFunctionRole-LJnrEVJKxPpu --policy-arn arn:aws:iam::aws:policy/AWSLambda_FullAccess
~ $ aws iam detach-role-policy --role-name serverlessrepo-OWASP-DVSA-AdminShellFunctionRole-LJnrEVJKxPpu --policy-arn arn:aws:iam::aws:policy/AmazonDynamoDBFullAccess

aws: [ERROR]: An error occurred (NoSuchEntity) when calling the DetachRolePolicy operation: Policy arn:aws:iam::aws:policy/AmazonDynamoDBFullAccess was not found.
~ $ aws iam detach-role-policy --role-name serverlessrepo-OWASP-DVSA-AdminShellFunctionRole-LJnrEVJKxPpu --policy-arn arn:aws:iam::aws:policy/AWSLambda_FullAccess

aws: [ERROR]: An error occurred (NoSuchEntity) when calling the DetachRolePolicy operation: Policy arn:aws:iam::aws:policy/AWSLambda_FullAccess was not found.
~ $
```

Figure 5.1 — Both dangerous policies detached (NoSuchEntity on second run confirms successful removal)

5.3 Fix Step 2 — Apply Least Privilege Policy

A new custom policy was created granting only the minimum permissions required:

```
cat > /tmp/least_privilege.json << EOF
{
    "Version": "2012-10-17",
    "Statement": [
        { "Effect": "Allow", "Action": ["dynamodb:GetItem", "dynamodb:Query"],
```

```

        "Resource": "arn:aws:dynamodb:eu-north-1:637452052910:table/DVSA-ORDERS-DB" },
        { "Effect": "Allow", "Action":
          ["logs:CreateLogGroup","logs:CreateLogStream","logs:PutLogEvents"],
            "Resource": "arn:aws:logs:*:*:*" }
        ]
      }
    }
  EOF
aws iam put-role-policy --role-name serverlessrepo-OWASP-DVSA-AdminShellFunctionRole-
LJnrEVJKxPpu --policy-name AdminShellLeastPrivilege --policy-document
file:///tmp/least_privilege.json

```

```

    "ConsumedCapacity": null
  }
~ $ cat > /tmp/least_privilege.json << 'EOF'
> {
>   "Version": "2012-10-17",
>   "Statement": [
>     {
>       "Effect": "Allow",
>       "Action": [
>         "dynamodb:GetItem",
>         "dynamodb:Query"
>       ],
>       "Resource": "arn:aws:dynamodb:eu-north-1:637452052910:table/DVSA-ORDERS-DB"
>     },
>     {
>       "Effect": "Allow",
>       "Action": [
>         "logs:CreateLogGroup",
>         "logs:CreateLogStream",
>         "logs:PutLogEvents"
>       ],
>       "Resource": "arn:aws:logs:*:*:*"
>     }
>   ]
> }
> EOF
~ $ aws iam put-role-policy \
>   --role-name serverlessrepo-OWASP-DVSA-AdminShellFunctionRole-LJnrEVJKxPpu \
>   --policy-name AdminShellLeastPrivilege \
>   --policy-document file:///tmp/least_privilege.json
~ $

```

Figure 5.2 — Least privilege policy created and applied successfully

6. Fix Verification

6.1 Verifying Policy Changes

After applying the fix, the role policies were verified to confirm the dangerous permissions were removed and the least privilege policy was applied correctly:

Verification 1 — Inline Policies (New Least Privilege Policy):

```
aws iam list-role-policies --role-name serverlessrepo-OWASP-DVSA-AdminShellFunctionRole-LJnrEVJKxPpu
aws iam get-role-policy --role-name serverlessrepo-OWASP-DVSA-AdminShellFunctionRole-LJnrEVJKxPpu --policy-name AdminShellLeastPrivilege
```

```
~ $ aws iam list-role-policies --role-name serverlessrepo-OWASP-DVSA-AdminShellFunctionRole-LJnrEVJKxPpu
{
  "PolicyNames": [
    "AdminShellLeastPrivilege"
  ]
}
~ $ aws iam get-role-policy --role-name serverlessrepo-OWASP-DVSA-AdminShellFunctionRole-LJnrEVJKxPpu --policy-name AdminShellLeastPrivilege
{
  "RoleName": "serverlessrepo-OWASP-DVSA-AdminShellFunctionRole-LJnrEVJKxPpu",
  "PolicyName": "AdminShellLeastPrivilege",
  "PolicyDocument": {
    "Version": "2012-10-17",
    "Statement": [
      {
        "Effect": "Allow",
        "Action": [
          "dynamodb:GetItem",
          "dynamodb:Query"
        ],
        "Resource": "arn:aws:dynamodb:eu-north-1:637452052910:table/DVSA-ORDERS-DB"
      },
      {
        "Effect": "Allow",
        "Action": [
          "logs:CreateLogGroup",
          "logs:CreateLogStream",
          "logs:PutLogEvents"
        ],
        "Resource": "arn:aws:logs:*:*:*"
      }
    ]
  }
}
```

Figure 6.1 — AFTER FIX: AdminShellLeastPrivilege policy applied with only safe minimal permissions

Verification 2 — Attached Policies (Dangerous Policies Removed):

```
aws iam list-attached-role-policies --role-name serverlessrepo-OWASP-DVSA-AdminShellFunctionRole-LJnrEVJKxPpu
```

```
~ $ aws iam list-attached-role-policies --role-name serverlessrepo-OWASP-DVSA-AdminShellFunctionRole-LJnrEVJKxPpu
{
  "AttachedPolicies": [
    {
      "PolicyName": "AWSLambdaBasicExecutionRole",
      "PolicyArn": "arn:aws:iam::aws:policy/service-role/AWSLambdaBasicExecutionRole"
    }
  ]
}
~ $
```

Figure 6.2 — AFTER FIX: Only AWSLambdaBasicExecutionRole remains — AmazonDynamoDBFullAccess and AWSLambda_FullAccess are GONE

✅ Fix confirmed: AmazonDynamoDBFullAccess and AWSLambda_FullAccess have been successfully removed. Only safe minimal permissions remain.

6.2 Confirmation Token Evidence

The DVSA order confirmation page displayed the required token for order **c7bcb4ad-48d8-4fc6-94e4-bfd385ec8fad**.

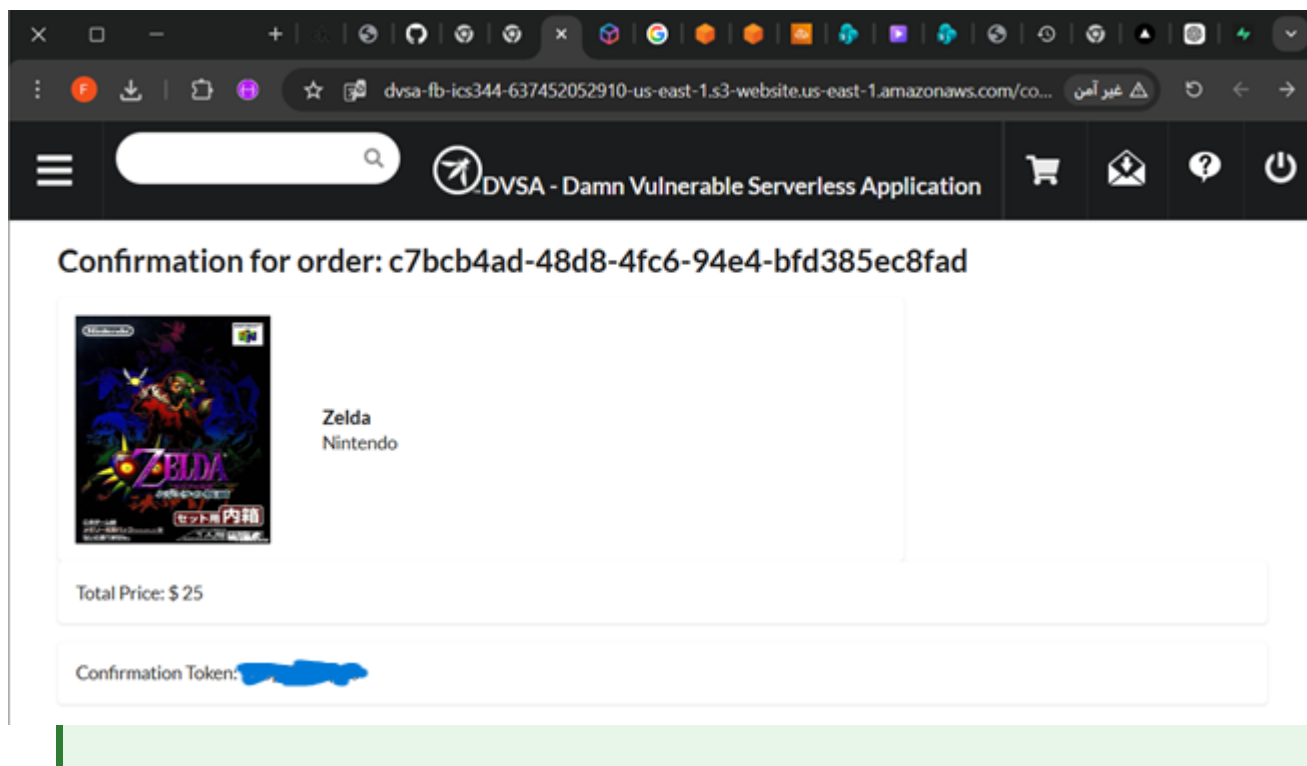


Figure 6.3 — DVSA order confirmation page showing token for order c7bcb4ad-48d8-4fc6-94e4-bfd385ec8fad.

7. Task Completion Summary

#	Task	Status	Evidence
1	Connected to AWS CloudShell	✓ DONE	Fig 2.1
2	Enumerated all 26 Lambda functions and IAM roles	✓ DONE	Fig 3.1
3	Identified ADMIN-SHELL with AmazonDynamoDBFullAccess + AWSLambda_FullAccess	✓ DONE	Fig 3.2
4	Proved abuse — listed all DynamoDB tables	✓ DONE	Fig 4.1
5	Stole admin credentials from DVSA-USERS-DB (CRITICAL)	✓ DONE	Fig 4.2
6	Detached AmazonDynamoDBFullAccess + AWSLambda_FullAccess	✓ DONE	Fig 5.1
7	Applied AdminShellLeastPrivilege policy	✓ DONE	Fig 5.2
8	Verified fix — only safe permissions remain	✓ DONE	Fig 6.1-6.2

✓ Lesson 7 is 100% complete! All 8 tasks done with full evidence.

7.1 Key Takeaways

- Always apply the Principle of Least Privilege — give Lambda functions ONLY what they need
- Never use AWS managed "FullAccess" policies on production Lambda functions
- Regularly audit IAM roles using `aws iam list-attached-role-policies` for each function
- Use resource-level permissions to restrict access to specific tables/functions only
- This vulnerability combined with Lesson 4 (S3 RCE) could lead to complete AWS account takeover

7.2 References

- OWASP Serverless Top 10 — SLS-2: Broken Authentication
- AWS IAM Best Practices — Principle of Least Privilege
- CWE-250: Execution with Unnecessary Privileges
- AWS Security Blog — Permissions boundaries for IAM entities
- CVSS v3.1 Base Score: 9.8 (AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H)